KITE Method

KITE stands for Kinematic Interpretable Trajectory Estimation. The method uses a vision-language model to predict interpretable driving intent and road geometry, then converts those language outputs into a physically plausible future trajectory with a bicycle model.

Problem Setting

For each KITScenes test sample, the model receives 12 camera images from the ego vehicle:

- front-left, front, and front-right cameras
- last 4 frames from each camera
- frame times: 0.6 s before now, 0.4 s before now, 0.2 s before now, and now

The output is a 5 second ego future trajectory:

- 25 waypoints
- sampled at 5 Hz
- local ego-frame coordinates, where x is forward and y is left

Core Idea

Directly asking Gemma for numeric waypoints is possible, but raw coordinates are brittle. KITE instead asks Gemma for structured, human-readable driving semantics:

- what the scene looks like
- where the drivable corridor is
- whether the lane is straight or curved
- whether to maintain speed, accelerate, or decelerate
- whether to steer left, steer right, or continue along the lane

The numeric trajectory is then produced by deterministic kinematics. This separates semantic reasoning from physical motion generation.

Compared Methods

KITE includes three methods.

1. Direct waypoints with ego history

   Gemma receives front-camera image history, the route instruction, and the past ego trajectory as text. It directly predicts 25 future x-y waypoints. No kinematic model is used.

2. Language actions with bicycle rollout

   Gemma receives front-camera image history, the route instruction, and current speed and heading from a Savitzky-Golay filter. It predicts acceleration and steering language for two time windows: the first 3 seconds and the last 2 seconds. A plain bicycle model converts those labels into waypoints.

3. Geometry actions with bicycle rollout

This is the recommended KITE method. Gemma predicts both driving actions and lane geometry. The geometry is converted into a small lane-following steering bias, so "steer straight" can still follow a curved road.

Gemma Output For The Recommended Method

The geometry-aware method asks Gemma to return JSON with an english object:

```
{
  "english": {
    "situational_awareness": "...",
    "lane_direction": "...",
    "lane_curvature_strength": "...",
    "ego_position_in_lane": "...",
    "drivable_corridor": "...",
    "trajectory_shape": "...",
    "acceleration_first_3s": "...",
    "steering_first_3s": "...",
    "acceleration_last_2s": "...",
    "steering_last_2s": "..."
  }
}
```

The first 15 rollout steps use the first 3 second action labels. The last 10 rollout steps use the last 2 second action labels.

Language To Physical Values

The code normalizes Gemma’s free-form text into canonical labels. Acceleration labels become longitudinal acceleration:

```
Label | Low-speed value | High-speed value
decelerate_strongly | -2.5 m/s^2 | -5.0 m/s^2
decelerate_slightly | -0.6 m/s^2 | -1.2 m/s^2
maintain_speed | 0.0 m/s^2 | 0.0 m/s^2
accelerate_slightly | 0.6 m/s^2 | 1.2 m/s^2
accelerate_strongly | 2.5 m/s^2 | 5.0 m/s^2
```

Steering labels become a base steering angle:

```
Label | Low-speed value | High-speed value
steer_left | 15 deg | 0.3 deg
steer_slightly_left | 5 deg | 0.1 deg
steer_straight | 0 deg | 0 deg
steer_slightly_right | -5 deg | -0.1 deg
steer_right | -15 deg | -0.3 deg
```

The low-speed table is used up to 60 km/h. The high-speed table is used above 60 km/h.

Geometry Steering Bias

For the geometry-aware method, KITE also converts lane geometry into a steering bias:

```
lane_direction + lane_curvature_strength + trajectory_shape -> delta_lane_bias
```

The sign comes from the predicted direction:

- left curve: positive steering bias

- right curve: negative steering bias
- straight: zero or very small steering bias

The magnitude comes mostly from the predicted curvature strength:

```
Curvature text | Low-speed bias | High-speed bias
sharp or high curve | 4.0 deg | 0.08 deg
medium, curve, or arc | 2.5 deg | 0.05 deg
slight or low curve | 1.25 deg | 0.025 deg
fallback weak curve | 0.75 deg | 0.015 deg
```

The final steering angle is:

```
delta_total_t = clip(delta_action_t + delta_lane_bias_t)
```

The clipping cap is 16 deg up to 60 km/h and 0.35 deg above 60 km/h. Steering is smoothed over the action window with an ease-in-out profile.

Bicycle Rollout

The initial state is estimated from the past ego trajectory using Savitzky-Golay derivatives:

```
v0 = sqrt(vx^2 + vy^2)
theta0 = atan2(vy, vx)
```

Then KITE rolls forward for 25 steps:

```
dt = 0.2 s
L = 2.7 m

v_{t+1} = max(0, v_t + a_t dt)
theta_{t+1} = theta_t + (v_{t+1} / L) tan(delta_total_t) dt
x_{t+1} = x_t + v_{t+1} cos(theta_{t+1}) dt
y_{t+1} = y_t + v_{t+1} sin(theta_{t+1}) dt
```

The final output is a 25-point local ego-frame trajectory.

Current Test Artifacts

The local three-method KITE run completed all 400 KITScenes test samples for each active method.

```
Method | Rows | Output
direct_waypoints_egohistory | 400 | direct waypoint baseline
language_actions_bicycle | 400 | language actions plus plain bicycle rollout
geometry_actions_bicycle | 400 | KITE geometry-aware bicycle rollout
```

The final geometry-aware submission artifact is:

```
outputs/kitscenes_test_front3_last4_geometry_allsteer_savgol_reasoning_geomkin_submission_final_with
```

Why This Helps

KITE makes the VLM responsible for semantic decisions and makes the kinematic model responsible for physical consistency. This gives a trajectory that is easier to inspect, easier to debug, and less dependent on Gemma producing perfectly calibrated numeric waypoints.